

Actividad final del módulo 2 - Programación Avanzada

"The backend is where the magic happens."

Objetivos de aprendizaje

- Aplicar principios de programación orientada a objetos y desarrollar código que se englobe en este paradigma de programación.
- Desarrollar aplicaciones web backend con el framework FastAPI, basado en Python.
- Usar diferentes verbos HTTP y diferentes técnicas habituales en el mundo backend, en arquitecturas REST (enviar payload, responder con el código HTTP adecuado a cada caso...).
- Desplegar aplicaciones backend en entornos locales.
- Crear código en Python usando la librería `requests` para interactuar con APIs.

Enunciado

A partir de la plantilla, se debe desarrollar una aplicación web backend en FastAPI, aplicando de forma integral los conceptos de programación orientada a objetos en Python que se han estudiado en el módulo.

La temática de la aplicación web será la gestión de una lista de tareas (funcionalidades mínimas indicadas a continuación).

La aplicación web debe:

- A nivel funcional, al menos, debe permitir añadir una nueva nota, obtener su contenido dado su id, marcar una nota como completada y obtener la lista de las notas caducadas (es decir, la fecha deadline de la nota es anterior a la fecha actual). La persistencia debe ser vía base de datos SQLite. De las notas, al menos, hay que guardar título, contenido y fecha de vencimiento (deadline).
- Implementar múltiples endpoints con diferentes verbos HTTP (GET, POST, PUT, DELETE, etc.). Al menos, hay que usar de forma justificada los verbos GET, POST y un método adicional a estos dos.
- En dicho código, usar clases que apliquen los conceptos de encapsulamiento, abstracción y otros principios fundamentales de programación orientada a objetos. Más allá de las clases básicas para definir la estructura de datos vía Pydantic o SQLAlchemy, se deben crear clases que puedan ser usadas en la parte de la aplicación web (por ejemplo, una clase `NoteManager` que permita limpiar ciertas palabras malsonantes del contenido de una nota, que permita hacer diferentes comprobaciones sobre la fecha deadline de la nota... aquí ya depende de la creatividad de cada uno). No hace falta que todos los métodos de las clases definidas tengan utilidad o se usen en la aplicación web, el objetivo es conseguir clases coherentes aplicando correctamente principios de POO.
- Cubrir la mayor variedad posible de técnicas y conceptos relacionados con APIs RESTful y backend en general, así como POO.
- De forma complementaria a la aplicación web, se debe crear un script en Python que utilice la librería `requests` para realizar peticiones HTTP a la API realizada. Recordemos que la API se puede desplegar en local y, por tanto, se puede (y se debe) probar todo esto en local, apuntando a nuestra API desplegada. Esto nos permitirá validar el correcto funcionamiento de la API y buscar posibles funcionamientos incoherentes en los endpoints. Para este ejercicio, en dicho script se debe desarrollar código que compruebe que las respuestas obtenidas coinciden con los resultados esperados: analizar el código HTTP recibido, body de la respuesta con los datos... según el caso de cada petición (por

ejemplo, en un POST, no suele tener tanto interés el body de la respuesta sino el código de respuesta; no ocurre lo mismo para las peticiones GET). Además, se debe probar un endpoint con una request con datos incorrectos, esperando un código de respuesta 400.

- Todo lo anterior son los requisitos mínimos requeridos, con lo que ya se puede obtener la máxima puntuación en caso de estar todo perfecto. Más allá de esto, se pueden añadir nuevos endpoints con nuevas funcionalidades y se valorarán positivamente siempre que sean correctos y que cumplan con los estándares estudiados de APIs REST, POO...

Entregables

- Un archivo `README.md` con una breve explicación de los endpoints implementados.
- Una carpeta `api`, dentro de la cual esté el código fuente de la aplicación web.
- Un archivo `test_python.py`, script en Python usando requests, debidamente comentado, que interactúe con la API (apuntando a localhost) y compruebe resultados.

Evaluación de la actividad

- Calidad del código general y uso adecuado de programación orientada a objetos. (40%)
- Variedad y correcta implementación de verbos HTTP y técnicas RESTful, ajustándose a los requisitos del enunciado. Funcionamiento de la API implementada. (40%)
- Calidad y correcto funcionamiento del script de prueba con requests. (20%)

Puntos extra

- Por aplicar funcionalidades adicionales en la API (autenticación, tareas asíncronas...), control de errores, etc, se podrá obtener hasta 1 punto extra (siendo igualmente la nota máxima un 10).

Fecha límite de entrega

Hasta dos semanas después de la finalización del módulo